Week 2 Structured Notes: Stack Memory, Variables, and Complexity Basics

1. Week 2 Main Goal

The goal of Week 2 is to understand how recursion works inside memory.

In Week 1, you learned what recursion is, how recursion stops, and how calling and returning phases work. Week 2 goes deeper and explains what happens inside the computer when a recursive function keeps calling itself.

By the end of Week 2, you should be able to explain:

- How recursion uses stack memory
- What an activation record is
- Why each recursive call has its own local variables
- How static and global variables behave differently
- Why recursion usually uses more memory than loops
- How to find simple time complexity and space complexity

Simple Week 2 question:

```
When a recursive function calls itself, where are those calls stored?
```

Simple answer:

```
They are stored in stack memory as separate activation records.
```

---

2. Quick Revision from Week 1

Before learning Week 2, remember these Week 1 concepts.

2.1 Recursion

Recursion means a function calls itself.

Example:

```c
void fun(int n) {
    if (n > 0) {
        fun(n - 1);
    }
}
```

Here, `fun()` calls itself, so it is a recursive function.

2.2 Base Case

The base case is the condition that stops recursion.

Example:

```c
if (n > 0)
```

When `n` becomes `0`, the condition becomes false, so the function stops calling itself.

2.3 Recursive Case

The recursive case is the part where the function calls itself again.

Example:

```c
fun(n - 1);
```

The input becomes smaller each time, so the function moves closer to the base case.

2.4 Calling Phase

The calling phase happens when recursive calls are going deeper.

Example:

```c
printf("%d ", n);
fun(n - 1);
```

Here, printing happens before the recursive call, so output appears during the calling phase.

2.5 Returning Phase

The returning phase happens when recursive calls finish and return back one by one.

Example:

```
fun(n - 1);
printf("%d ", n);
```

Here, printing happens after the recursive call, so output appears during the returning phase.

---

3. Why Week 2 Is Important

Many beginners think recursion is just a function calling itself. That is only the surface.

The real question is:

```
How does the computer remember all those unfinished function calls?
```

The answer is:

```
The computer stores them in stack memory.
```

When a recursive function calls itself, the current call is not finished yet. It must wait until the next call finishes. So the computer keeps each unfinished function call in the stack.

This is why recursion needs more memory than a normal loop.

---

4. Basic Memory Areas in a C Program

A C program uses different parts of memory. For Week 2, you only need a beginner-level understanding.

Memory Area	Simple Meaning	Example Use
Code section	Stores program instructions	Function code
Stack	Stores function calls and local variables	Recursive calls
Heap	Used for dynamic memory	`malloc()` memory
Global/static area	Stores global and static variables	Global variables, static variables

For recursion, the most important memory area is:

```
Stack memory
```

---

5. Stack Memory

Stack memory is the part of memory used to manage function calls.

Whenever a function is called, a new memory block is created on the stack. When the function finishes, that memory block is removed.

5.1 Stack Works Like a Pile

Think of the stack like a pile of plates.

If you place plates one by one:

```
Plate 1
Plate 2
Plate 3
```

The last plate you place on top is the first plate you remove.

This rule is called:

```
LIFO: Last In, First Out
```

5.2 LIFO in Recursion

For this recursive call:

```
fun(3);
```

The function calls may happen like this:

```
fun(3)
fun(2)
fun(1)
fun(0)
```

The last call is `fun(0)`, so it finishes first.

Returning order:

```
fun(0) finishes first
fun(1) finishes next
fun(2) finishes next
fun(3) finishes last
```

That is stack behavior.

---

6. Activation Record

An activation record is the memory block created for one function call.

It is also called a stack frame.

Simple meaning:

```
One function call = one activation record
```

Each activation record stores information needed for that function call.

It may store:

- Function parameter values
- Local variables
- Return address
- Temporary values
- Information needed to continue after the function returns

---

7. Stack Frame

A stack frame is another name for an activation record.

When a function is called, a stack frame is created.

When the function finishes, its stack frame is removed.

Example:

```c
#include <stdio.h>

void fun(int n) {
    if (n > 0) {
        printf("%d ", n);
        fun(n - 1);
    }
}

int main() {
    fun(3);
    return 0;
}
```

Function call:

```
fun(3);
```

---

8. Stack Trace for `fun(3)`

Step 1: Program starts from `main()`

```
main()
```

Step 2: `main()` calls `fun(3)`

```
fun(3)
main()
```

Step 3: `fun(3)` calls `fun(2)`

```
fun(2)
fun(3)
main()
```

Step 4: `fun(2)` calls `fun(1)`

```
fun(1)
fun(2)
```

```
fun(3)
main()
```

Step 5: `fun(1)` calls `fun(0)`

```
fun(0)
fun(1)
fun(2)
fun(3)
main()
```

Step 6: `fun(0)` stops

When `n = 0`, the condition `n > 0` becomes false. So `fun(0)` does not call another function.

Now stack frames are removed one by one.

Step 7: Returning starts

After `fun(0)` finishes:

```
fun(1)
fun(2)
fun(3)
main()
```

After `fun(1)` finishes:

```
fun(2)
fun(3)
main()
```

After `fun(2)` finishes:

```
fun(3)
main()
```

After `fun(3)` finishes:

```
main()
```

After `main()` finishes, the program ends.

---

9. Main Idea of Stack in Recursion

The stack stores unfinished function calls.

When recursion goes deeper, stack frames are added.

When recursion returns, stack frames are removed.

Simple rule:

```
Recursive calls add stack frames.
Returning removes stack frames.
```

---

10. Local Variables in Recursion

A local variable is a variable declared inside a function.

Example:

```c
int fun(int n) {
    int x = 10;
    return x;
}
```

Here, `x` is a local variable.

In recursion, each function call gets its own copy of local variables.

This is a very important idea.

Simple rule:

```
Each recursive call has its own local variables.
```

---

11. Example: Local Variable in Recursive Sum

Look at this code:

```
int sumLocal(int n) {
    if (n > 0) {
        return sumLocal(n - 1) + n;
    }
    return 0;
}
```

Function call:

```
sumLocal(3);
```

The recursive calls are:

```
sumLocal(3)
sumLocal(2)
sumLocal(1)
sumLocal(0)
```

Each call has its own `n`:

```
sumLocal(3): n = 3
sumLocal(2): n = 2
sumLocal(1): n = 1
sumLocal(0): n = 0
```

The values do not overwrite each other.

That means `n = 3`, `n = 2`, `n = 1`, and `n = 0` are stored in different stack frames.

---

12. Stack View for `sumLocal(3)`

When the recursion reaches the deepest point, the stack looks like this:

```
sumLocal(0): n = 0
sumLocal(1): n = 1
sumLocal(2): n = 2
```

```
sumLocal(3): n = 3
main()
```

Each function call has its own separate `n`.

This is why recursive functions can return the correct answer.

---

13. Returning Phase of `sumLocal(3)`

Code:

```
int sumLocal(int n) {
    if (n > 0) {
        return sumLocal(n - 1) + n;
    }
    return 0;
}
```

Call:

```
sumLocal(3);
```

The function first goes down until `n = 0`.

Then it returns values back.

Returning trace:

```
sumLocal(0) returns 0
sumLocal(1) returns sumLocal(0) + 1 = 0 + 1 = 1
sumLocal(2) returns sumLocal(1) + 2 = 1 + 2 = 3
sumLocal(3) returns sumLocal(2) + 3 = 3 + 3 = 6
```

Final answer:

```
6
```

---

14. Why Local Variables Do Not Overwrite Each Other

This is a common beginner confusion.

You may think:

```
If the function keeps calling itself, will the new n replace the old n?
```

Answer:

```
No. Each function call gets a separate stack frame.
```

So:

```
sumLocal(3) has its own n
sumLocal(2) has its own n
sumLocal(1) has its own n
sumLocal(0) has its own n
```

They are separate copies.

That is why recursion can remember earlier values during the returning phase.

---

15. Static Variables in Recursion

A static variable is created only once.

Example:

```
static int x = 0;
```

Unlike local variables, a static variable is not created again for every function call.

Simple rule:

```
A static variable has only one copy.
```

All recursive calls share the same static variable.

---

16. Example: Static Variable in Recursion

Code:

```
int funStatic(int n) {
    static int x = 0;

    if (n > 0) {
        x++;
        return funStatic(n - 1) + x;
    }

    return 0;
}
```

Call:

```
funStatic(3);
```

Calling Phase

```
funStatic(3): x becomes 1
funStatic(2): x becomes 2
funStatic(1): x becomes 3
funStatic(0): returns 0
```

At the deepest point, `x` is now `3`.

Returning Phase

Because `x` is static, all calls use the same `x`.

```
funStatic(1) returns 0 + 3 = 3
funStatic(2) returns 3 + 3 = 6
funStatic(3) returns 6 + 3 = 9
```

Final answer:

```
9
```

---

17. Why Static Variable Output Can Be Confusing

Many beginners expect `funStatic(3)` to return `6`, but it returns `9`.

Why?

Because `x` is not separate for each function call.

There is only one `x`.

All calls share it.

By the time returning begins, `x` has already become `3`. So each returning call uses `x = 3`.

This is why static variables can make recursion harder to understand.

---

18. Static Variable Warning

A static variable remembers its value even after the function finishes.

Example:

```
printf("%d\n", funStatic(3));
printf("%d\n", funStatic(3));
```

The second call may not behave like the first call because the static variable may still remember its old value.

Simple warning:

```
Static variables can remember old values across function calls.
```

So use static variables carefully in recursion.

---

19. Global Variables in Recursion

A global variable is declared outside all functions.

Example:

```c
int x = 0;

int funGlobal(int n) {
    if (n > 0) {
        x++;
        return funGlobal(n - 1) + x;
    }
    return 0;
}
```

Here, `x` is a global variable.

A global variable has only one copy.

All functions can access it.

Simple rule:

```
Global variables are shared by the whole program.
```

---

20. Global Variables vs Static Variables

Static variables and global variables are similar because both have only one copy.

But they are not exactly the same.

Variable Type	Where It Is Declared	Number of Copies	Who Can Use It?
Static variable inside function	Inside a function using `static`	One	Usually only that function
Global variable	Outside all functions	One	Many functions can use it

Both can make recursion harder to trace because they are shared.

---

21. Local vs Static vs Global Variables

Variable Type	Memory Behavior	Number of Copies	Risk or Warning
Local variable	Created inside each function call	Many copies, one per call	Safe, but uses stack space
Static variable	Created once and shared across calls	One copy	Can remember old values
Global variable	Declared outside functions and shared globally	One copy	Can make recursion harder to understand

Simple summary:

```
Local variables belong to each call.
Static variables are shared across calls.
Global variables are shared across the program.
```

---

22. Time Complexity Basics

Time complexity explains how running time grows when input size grows.

Simple question:

```
How many steps or function calls happen?
```

If the number of calls grows with `n`, the time complexity is often `O(n)`.

---

23. Counting Function Calls

Code:

```c
int sumLocal(int n) {
    if (n > 0) {
        return sumLocal(n - 1) + n;
    }
    return 0;
}
```

Call:

```
sumLocal(3);
```

Function calls:

```
sumLocal(3)
sumLocal(2)
sumLocal(1)
sumLocal(0)
```

Number of calls:

```
4 calls
```

For `n = 3`, there are about `n + 1` calls.

For `n = 100`, there are about `101` calls.

So the time complexity is:

```
O(n)
```

Why?

Because the number of function calls grows linearly with `n`.

---

24. Space Complexity Basics

Space complexity explains how much extra memory is used.

In recursion, the main extra memory comes from stack frames.

For this call:

```
sumLocal(3);
```

The stack frames are:

```
sumLocal(3)
sumLocal(2)
sumLocal(1)
sumLocal(0)
```

There are about `n + 1` stack frames.

So the space complexity is:

```
O(n)
```

Why?

Because the stack grows as `n` grows.

---

25. Simple Linear Recursion

A simple linear recursion is a recursive function that makes only one recursive call each time.

Example:

```
int sumLocal(int n) {
    if (n > 0) {
        return sumLocal(n - 1) + n;
    }
    return 0;
}
```

Each call makes only one more call:

```
sumLocal(3) calls sumLocal(2)
sumLocal(2) calls sumLocal(1)
sumLocal(1) calls sumLocal(0)
```

So this is linear recursion.

Complexity:

```
Time complexity: O(n)
Space complexity: O(n)
```

---

26. Recursion vs Loop

A recursive function and a loop can sometimes solve the same problem.

Recursive Version

```
int sumRecursive(int n) {
    if (n == 0) {
        return 0;
    }
    return sumRecursive(n - 1) + n;
}
```

Loop Version

```
int sumLoop(int n) {
    int sum = 0;

    for (int i = 1; i <= n; i++) {
        sum = sum + i;
    }

    return sum;
}
```

For `n = 5`, both return:

```
15
```

But memory usage is different.

Version	Time Complexity	Space Complexity	Reason
Recursive version	O(n)	O(n)	Creates many stack frames
Loop version	O(n)	O(1)	Reuses the same variables

Important lesson:

```
A loop often uses less memory than recursion.
```

---

27. Why Recursion Usually Uses More Memory Than a Loop

In a loop, the same function keeps running.

Example:

```
for (int i = 1; i <= n; i++) {
    sum = sum + i;
}
```

The loop does not create a new function call every time.

But recursion creates a new function call each time.

Example:

```
sumRecursive(n - 1);
```

Each recursive call creates a new stack frame.

So recursion usually uses more memory.

Simple comparison:

```
Loop repeats work using the same stack frame.
Recursion repeats work by creating new stack frames.
```

---

28. Stack Overflow

Stack overflow happens when the stack becomes full.

This can happen when recursion goes too deep.

Example problem:

```c
void wrong(int n) {
    printf("%d ", n);
    wrong(n + 1);
}
```

This function has no base case.

It keeps calling itself forever until the stack is full.

Result:

```
Program crash
```

Simple rule:

```
Too many recursive calls can cause stack overflow.
```

---

29. Common Beginner Mistakes in Week 2

Mistake 1: Thinking There Is Only One `n`

Wrong thinking:

```
Every recursive call uses the same n.
```

Correct thinking:

```
Each recursive call has its own n because each call has its own stack frame.
```

Mistake 2: Forgetting That Static Variables Are Shared

Wrong thinking:

```
A static variable resets in every recursive call.
```

Correct thinking:

```
A static variable has one copy and is shared across calls.
```

Mistake 3: Thinking Recursion and Loops Always Use Same Memory

Wrong thinking:

```
If recursion and loop both run n times, memory must be the same.
```

Correct thinking:

```
Both may have O(n) time, but recursion usually has O(n) stack space while a loop
often has O(1) space.
```

Mistake 4: Only Tracing Output

Wrong habit:

```
Only predicting the printed answer.
```

Correct habit:

```
Trace the stack frames and variable values also.
```

---

30. Full Practice Program for Week 2

```c
#include <stdio.h>

int sumLocal(int n) {
    if (n > 0) {
        return sumLocal(n - 1) + n;
    }
    return 0;
}

int funStatic(int n) {
    static int x = 0;

    if (n > 0) {
        x++;
```

```
        return funStatic(n - 1) + x;
    }

    return 0;
}

int main() {
    printf("sumLocal(3) = %d\n", sumLocal(3));
    printf("funStatic(3) = %d\n", funStatic(3));

    return 0;
}
```

Expected output:

```
sumLocal(3) = 6
funStatic(3) = 9
```

Explanation:

```
sumLocal(3) returns 6 because each call has its own n.
funStatic(3) returns 9 because all calls share the same x.
```

---

31. Dry Run: `sumLocal(3)`

Code:

```
int sumLocal(int n) {
    if (n > 0) {
        return sumLocal(n - 1) + n;
    }
    return 0;
}
```

Call:

```
sumLocal(3);
```

Calling Phase

```
sumLocal(3) waits for sumLocal(2) + 3
sumLocal(2) waits for sumLocal(1) + 2
sumLocal(1) waits for sumLocal(0) + 1
sumLocal(0) returns 0
```

Returning Phase

```
sumLocal(1) = 0 + 1 = 1
sumLocal(2) = 1 + 2 = 3
sumLocal(3) = 3 + 3 = 6
```

Final answer:

```
6
```

---

32. Dry Run: `funStatic(3)`

Code:

```c
int funStatic(int n) {
    static int x = 0;

    if (n > 0) {
        x++;
        return funStatic(n - 1) + x;
    }

    return 0;
}
```

Call:

```
funStatic(3);
```

Calling Phase

```
funStatic(3): x = 1
funStatic(2): x = 2
funStatic(1): x = 3
funStatic(0): returns 0
```

Returning Phase

```
funStatic(1): returns 0 + 3 = 3
funStatic(2): returns 3 + 3 = 6
funStatic(3): returns 6 + 3 = 9
```

Final answer:

```
9
```

---

33. Complexity Summary Table

Pattern	Time Complexity	Space Complexity	Reason
Simple linear recursion	O(n)	O(n)	Makes about n calls and stores stack frames
Equivalent loop	O(n)	O(1)	Repeats n times but reuses the same stack frame

---

34. Important Week 2 Terms

Stack

Memory area that stores function calls.

Stack Frame

The memory block created for one function call.

Activation Record

Another name for a stack frame.

Local Variable

A variable created inside a function. Each recursive call gets its own copy.

Static Variable

A variable created once and shared across function calls.

Global Variable

A variable declared outside all functions and shared by the whole program.

Time Complexity

A way to describe how runtime grows as input size grows.

Space Complexity

A way to describe how memory usage grows as input size grows.

Stack Overflow

An error that happens when too many function calls fill the stack.

---

35. Week 2 Simple Mental Model

Think of recursion like opening boxes.

You open Box 3. Inside it, there is Box 2. Inside that, there is Box 1. Inside that, there is Box 0.

You cannot close Box 3 until Box 2 is closed.

You cannot close Box 2 until Box 1 is closed.

You cannot close Box 1 until Box 0 is closed.

So the closing happens in reverse order.

```
Box 0 closes first
Box 1 closes second
Box 2 closes third
Box 3 closes last
```

That is how recursive calls return from the stack.

---

36. How to Study Week 2 Properly

Do not only run the program.

Use this method:

1. Read the code.
2. Find the base case.
3. Find the recursive call.
4. Write the call chain.
5. Draw the stack frames.
6. Write local variable values for each call.
7. Trace the returning phase.
8. Count the number of calls.
9. Find time complexity.
10. Find space complexity.
11. Compare with a loop version if possible.

---

37. Week 2 Practice Questions

Question 1

What is stack memory?

Question 2

What is an activation record?

Question 3

Why does each recursive call have its own local variable?

Question 4

What is the difference between a local variable and a static variable?

Question 5

Why can static variables produce unexpected results in recursion?

Question 6

Why does recursion usually use more memory than loops?

Question 7

What is the time complexity of this function?

```
int fun(int n) {
    if (n > 0) {
        return fun(n - 1) + n;
    }
    return 0;
}
```

Question 8

What is the space complexity of the same function?

Question 9

Draw the stack for:

```
fun(3);
```

Question 10

Why does a loop version usually have `O(1)` space?

---

38. Week 2 Mastery Checklist

Before moving to Week 3, you should be able to do these:

- Explain stack memory in simple words
- Explain what an activation record is
- Draw stack frames for `fun(3)`
- Explain why local variables do not overwrite each other
- Explain the difference between local, static, and global variables
- Trace `sumLocal(3)` correctly
- Trace `funStatic(3)` correctly
- Explain why `funStatic(3)` gives `9`
- Count function calls in simple recursion
- Find `O(n)` time complexity for linear recursion
- Find `O(n)` space complexity for linear recursion
- Explain why loop space is often `O(1)`
- Explain why recursion can cause stack overflow

---

39. Week 2 Final Summary

Week 2 explains how recursion works in memory.

When a recursive function calls itself, each call is stored on the stack as a separate activation record. Each activation record has its own parameter values and local variables. That is why local variables from different recursive calls do not overwrite each other.

Static variables and global variables behave differently. They have only one copy, so all recursive calls share the same value. This can make recursion harder to trace and may produce unexpected results if the function is called multiple times.

Simple linear recursion usually has `O(n)` time complexity because it makes about `n` function calls. It also has `O(n)` space complexity because those calls are stored on the stack. A loop can do the same repeated work with `O(n)` time but usually only `O(1)` extra space.

The most important Week 2 rule is:

```
Do not only trace the output. Trace the stack and variables also.
```

Week 1 taught what recursion does.

Week 2 teaches where recursion lives while it is running.